Automated Research Report Generator Using N8N

Project Title: Automated Research Report Generation System using N8N
Student Name: Vishrut Bharti
Student ID: 23095121
Course Name: Powers of AI
Date: March 08, 2026

Drive link :Video Link **Video Demonstration**

Introduction

In the current era of abundant digital information, individuals seeking to conduct systematic research encounter significant obstacles in managing and synthesizing vast resources. The volume of online content has grown exponentially, making it extremely challenging for researchers, students, and professionals to efficiently locate, assess, and extract meaningful insights from an ever-increasing pool of online resources. This environment fosters information overload, with critical facts often hidden among irrelevant content. Moreover, the manual research process which includes searching, verifying sources, extracting key findings, organizing results, and properly citing sources remains exceedingly time consuming and susceptible to human error and inconsistency. In an academic or business setting, this inefficiency directly hampers productivity, limits the depth of analysis, and increases the risk of oversight.

The motivation behind this project is to automate the core steps in the research process for quicker, higher quality, and more uniform output. Automating research not only saves considerable time but also enables parallelization, allowing multiple research topics to be explored simultaneously, and frees human researchers to focus on detailed analysis and critical thinking, rather than mechanical data collection and processing. By constructing an automated system that can gather, filter, synthesize, and deliver complete research reports, this approach revolutionizes how information is handled for both individual and organizational decision making.

Workflow Design

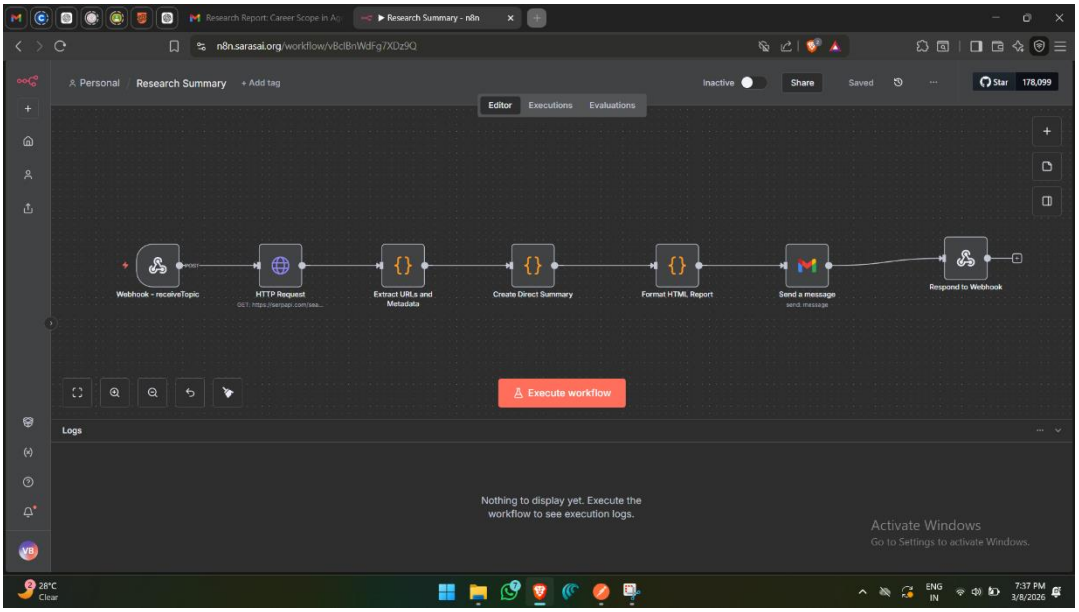
The solution is built using N8N, a modern no code automation platform, which coordinates multiple external API services and in-house logic nodes to create an end-to-end research pipeline. The logic begins with a webhook trigger node, which acts as the system's interface, listening for incoming HTTP POST requests with the research topic encapsulated in a straightforward JSON format. Once activated, this workflow immediately launches a web search using SerpAPI, which is connected via an HTTP request node with securely managed API credentials.Postman was utilized for testing and triggering the webhook node. It sent HTTP POST requests containing the topic payload in JSON format. Its intuitive interface facilitated the construction of request bodies, inspection of responses, and debugging of webhook integration prior to the deployment of live API endpoints.

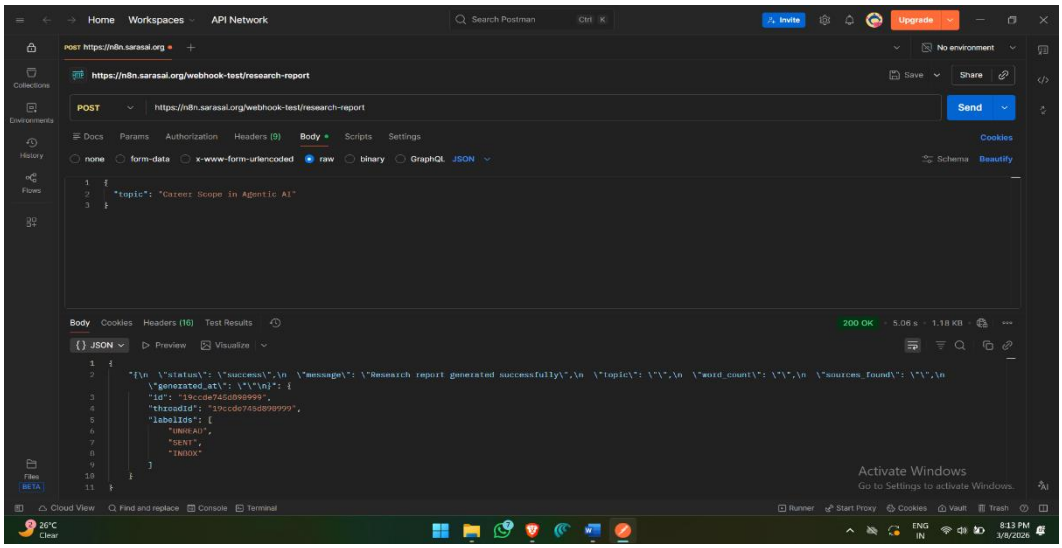
SerpAPI's response composed of a structured list of organic results and news articles is passed to a JavaScript code node that executes comprehensive data cleansing and validation. This step ensures that only relevant, accessible web pages are considered for further processing, effectively eliminating privacy, login, cached, and file download URLs. The code also employs strong deduplication logic, filters based on URL patterns, and compiles a refined pool of sources with their corresponding metadata for traceability.

Next, the workflow utilizes another JavaScript node, responsible for synthesizing the extracted snippets into a logically structured research summary. Rather than relying on external AI text generation APIs, which could incur costs or privacy risks, this node employs intelligent content aggregation and organization logic, grouping findings into standardized sections: an introduction, key takeaways, a source diversity overview, thematic analyses across current state, trends, challenges, industry impact, and a concise conclusion. The summary is formatted using Markdown syntax for clarity and then converted into a polished HTML report using another processing node. The HTML is styled with inline CSS to ensure compatibility and accessibility across different email clients and devices.

The finished report is immediately delivered to stakeholders or users via a Gmail integration node, sending a professional HTML email with all critical findings and proper citations. Simultaneously, a webhook response returns status and metadata to the calling API or user. Below is a simplified schematic representing this workflow:

```
Webhook Trigger → SerpAPI Search → URL Extraction & Validation (JS)
→ Summary Generation (JS)
→ HTML Formatting (JS) → Gmail Delivery → Webhook Response
```





The agent logic is fundamentally governed by JavaScript code level prompts embedded in the summary generation node. These prompts dictate how evidence is aggregated, the minimum number of findings required, the structural template to be used, and how reliability and deduplication are enforced, making the process both transparent and reproducible.

Challenges & Solutions

One of the initial technical hurdles involved handling API authentication and rate limits with SerpAPI. The workflow encountered errors due to improper credential management and exceeded thresholds. This was resolved by storing credentials in N8N's encrypted store and enabling error-handling logic to mitigate authentication failures and inform users of issues.

Constant monitoring ensures that API usage stays within the allocated quota, and future improvements may include caching to avoid redundant searches.

Filtering and validating URLs was another pain point. Initial web searches returned numerous invalid results such as login portals, cached copies, and downloadable files, which disrupted downstream processing. A robust validation function was implemented to systematically exclude irrelevant or unreliable sources, greatly reducing bad URLs and improving both coverage and quality of the final reports.

A crucial design constraint was avoiding expensive or opaque AI summarization APIs. The system, therefore, leverages hand crafted JavaScript logic to extract, deduplicate, and systematically organize snippets into meaningful summaries. This solution is transparent and cost effective, with the trade-off of sometimes lacking the depth and nuance an LLM might provide. Nevertheless, it ensures that findings are clearly traceable to original sources and that outputs are deterministic.

A significant formatting challenge was encountered in rendering HTML reports consistently across diverse email clients. Since platforms like Gmail and Outlook interpret styles in a nonstandard way, the report generator was refactored to use all inline CSS, table-based layouts for metadata, and universal, web safe fonts and color schemes. Each iteration was extensively tested for accessibility, rendering speed, and cross-platform reliability, resulting in a visually professional and readable output that remains consistent for all users.

Robustness was improved by layering in error handling strategies at each node, including input validation, null checking, defensive coding practices for network errors, and informative error reporting back to users. This ensures graceful degradation, so even if partial failures occur, users receive clear feedback and a best-effort report rather than a failed or misleading response.

Finally, performance bottlenecks were addressed by optimizing loops, limiting processed sources, aggregating strings and HTML efficiently, and using early exits and O(1) data structures for deduplication. These efforts reduced the report generation and delivery time to about 15-25 seconds per request, making the system viable for live use.

Conclusion

This project demonstrates the effectiveness and practicality of no-code workflow automation for complex cognitive tasks like research report generation. The designed N8N workflow transforms a simple topic query into a comprehensive, professionally formatted research deliverable, using only validated web sources and intelligently structured output formation, with no reliance on costly AI APIs. The architecture is robust, fault tolerant, and highly scalable, capable of processing multiple requests in parallel and delivering reliable output within seconds.

Potential future improvements include integrating caching for repeated queries, expanding to alternate APIs for greater search diversity, incorporating basic semantic analysis for deeper insights, and building modular plugins for destination outputs (e.g., Google Docs, PDFs, or Slack). By further optimizing both content aggregation and downstream delivery mechanisms, the system can become an essential productivity tool for academics, businesses, and any knowledge worker operating in today's information-rich world.